HTTP Test (with Python Socket)

컴퓨터네트워크 과제

TCP 기반 Server / Client HTTP 통신 구현

□ 목적

- TCP 기반 소켓 통신을 활용하여 HTTP Request / Response 동작 원리를 이해한다.

- Client 가 HTTP 형식의 요청 메시지를 직접 생성하고, Server 가 요청에 따라 응답 메시지를 구성하도록 구현한다.

- Wireshark 를 사용하여 구현한 HTTP 패킷 형식을 확인한다.

□ 목표

1. TCP 기반 Server, Client 프로그램 구현

2. Client 에서 GET / HEAD / POST / PUT Request 요청

3. Server 에서 Client Request 에 따라 HTTP Response 구성

4. Method-Response 조합 Case 5 개 이상 수행

5. Wireshark 로 HTTP format 캡쳐

6. Server 는 UTM Ubuntu VM, Client 는 macOS 에서 분리 실행

□ 실행 방법

GitHub 에서 소스 코드 다운로드

```
$ git clone https://github.com/hohyun-nam/computernetwork_http.git
$ cd computernetwork_http
```

Server 실행

```
$ cd server
$ python3 server.py
```

Client 실행

```
$ cd client
$ python3 client.py --host 192.168.64.2 --port 8080
```

Server 의 IP 주소가 다르면 --host 값을 실제 IP 주소로 변경한다.

□ 유의사항

- Server 를 먼저 실행한 뒤 Client 를 실행한다.

- Client 메뉴에서 a 를 입력하면 전체 Test Case 를 순서대로 실행한다.

- DELETE 200 Case 는 PUT /sample.txt 수행 후 파일이 생성된 상태에서 정상 동작한다.

- Wireshark 필터는 tcp.port == 8080 또는 http 를 사용한다.

□ 개발환경

구분	환경
Server	UTM Ubuntu VM / Python 3 / server.py
Client	macOS / Python 3 / client.py
통신 방식	TCP Socket
Port	8080

□ Test Case 설계

Case	Method	Path	Request 내용	Response
1	GET	/	기본 페이지 요청	200 OK
2	GET	/abc	존재하지 않는 경로 요청	404 Not Found
3	POST	/message	Body: hello professor	201 Created
4	POST	/message	빈 Body 전송	400 Bad Request
5	PUT	/sample.txt	작은 파일 데이터 업로드	200 OK
6	PUT	/large.txt	제한 크기 초과 데이터 업로드	413 Payload Too Large
7	DELETE	/sample.txt	업로드된 파일 삭제	200 OK

8	DELETE	/ghost.txt	없는 파일 삭제 요청	404 Not Found
9	DELETE	/protected.txt	보호 파일 삭제 요청	403 Forbidden
10	HEAD	/	기본 페이지 Header 요청	200 OK

□ 구현 상황

과제 요구사항	적용 여부	구현 파일
TCP 기반 Server 프로그램 작성	O	server/server.py
TCP 기반 Client 프로그램 작성	O	client/client.py
GET Request / Response 구현	O	client.py / server.py
HEAD Request / Response 구현	O	client.py / server.py
POST Request / Response 구현	O	client.py / server.py
PUT Request / Response 구현	O	client.py / server.py
5 개 이상 Case 수행	O	총 10 개 Case
Wireshark HTTP format 캡쳐 가능	O	TCP 8080 사용

□ TCP 프로그래밍

Server 는 TCP 소켓을 생성하고 0.0.0.0:8080 에 바인딩한 뒤 Client 연결을 기다린다.

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.bind((args.host, args.port))
    server.listen(5)
    conn, addr = server.accept()
```

Client 는 각 요청마다 TCP 소켓을 생성하고 Server IP 와 Port 로 연결한 후 HTTP Request 를 전송한다.

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
```

```
    client.connect((host, port))
    client.sendall(request_bytes)
    response = receive_response(client)
```

□ HTTP Protocol

Client 는 Request Line, Header, Body 를 HTTP/1.1 형식에 맞게 직접 구성한다. GET 과 HEAD 요청은 Server 의 www 폴더 안 실제 파일 경로와 연결된다.

```
GET / HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
```

POST 와 PUT 처럼 Body 가 있는 요청은 Content-Length 와 Content-Type Header 를 포함한다.

```
POST /message HTTP/1.1
Content-Length: 15
Content-Type: text/plain

hello professor
```

Server 는 Request 를 Method 별로 분기하고, 상태 코드와 Header, Body 를 포함한 Response 를 생성한다.

GET 과 HEAD 는 요청 Path 를 server/www 폴더 안의 파일 경로로 바꾸고, 파일이 존재하면 200 OK, 없으면 404 Not Found 를 반환한다.

```
if method == "GET":
    response = handle_get(path)
elif method == "HEAD":
    response = handle_head(path)
elif method == "POST":
    response = handle_post(path, body)
elif method == "PUT":
    response = handle_put(path, body)
```

□ CASE 설명

CASE1: GET / 요청은 기본 경로이므로 server/www/index.html 을 반환하며 200 OK 가 발생한다.

CASE2: GET /abc 요청은 server/www/abc 파일 또는 server/www/abc/index.html 이 없기 때문에 404 Not Found 가 발생한다.

CASE3: POST /message 에 Body 를 전송하면 201 Created 가 발생한다.

CASE4: POST /message 에 빈 Body 를 전송하면 400 Bad Request 가 발생한다.

CASE5: PUT /sample.txt 에 작은 데이터를 전송하면 파일 저장 후 200 OK 가 발생한다.

CASE6: PUT /large.txt 에 큰 데이터를 전송하면 413 Payload Too Large 가 발생한다.

CASE7: DELETE /sample.txt 는 업로드된 파일을 삭제하고 200 OK 가 발생한다.

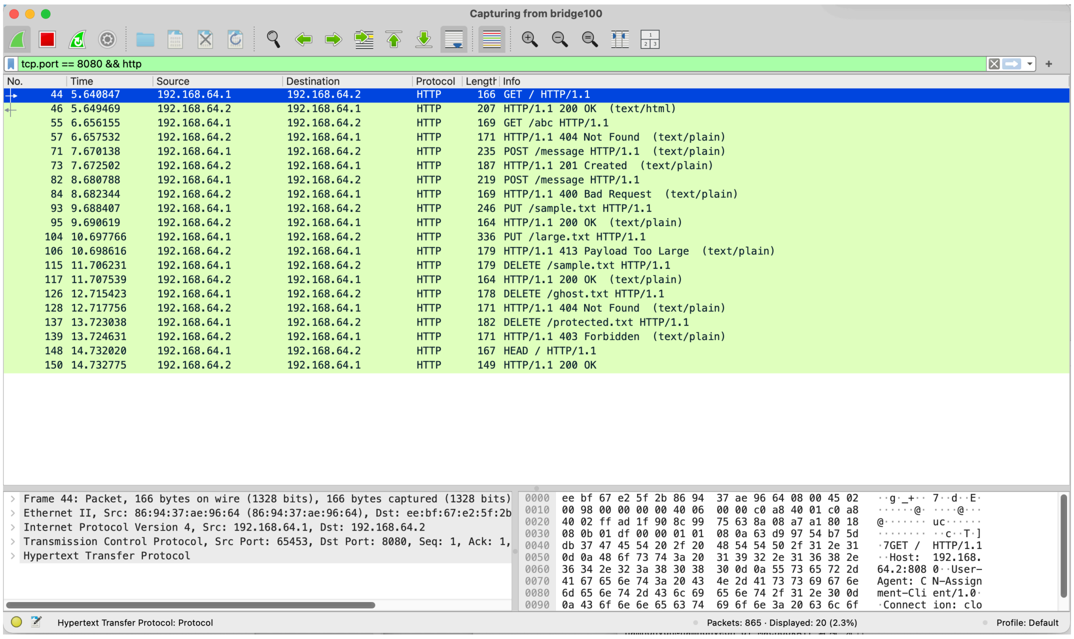
CASE8: DELETE /ghost.txt 는 없는 파일이므로 404 Not Found 가 발생한다.

CASE9: DELETE /protected.txt 는 보호 파일이므로 403 Forbidden 이 발생한다.

CASE10: HEAD / 요청은 server/www/index.html 이 존재하므로 Header 만 반환하며 200 OK 가 발생한다.

□ WireShark 캡쳐 내용

캡쳐 필터: tcp.port == 8080 && http



□ 실행 화면

Server 실행 화면

```
hohyun@hohyun-pc:~/server$ python3 server.py
HTTP Server Started (0.0.0.0:8080)
```

Client 실행 화면

```
namhohyun@namhohyeon-ui-MacBookAir client % python3 client.py

========== MENU ==========
1. GET 200
2. GET 404
3. POST 201
4. POST 400
5. PUT 200
6. PUT 413
7. DELETE 200
8. DELETE 404
9. DELETE 403
10. HEAD 200
a. RUN ALL
0. EXIT
Select :
```

□ 실행 화면

CASE1

Server

```
========== REQUEST HEADER ==========
GET / HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

========== RESPONSE ==========
HTTP/1.1 200 OK
Content-Length: 58
Content-Type: text/html
Connection: close

<html>
<body>
<h1>HTTP Server Running</h1>
</body>
</html>
```

Client

```
========== 1. GET / -> 200 OK ==========
[Client -> Server]
GET / HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 200 OK
Content-Length: 58
Content-Type: text/html
Connection: close

<html>
<body>
<h1>HTTP Server Running</h1>
</body>
</html>
```

CASE2

Server

```
Client Connected : ('192.168.64.1', 50195)

========== REQUEST HEADER ==========
GET /abc HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

========== RESPONSE ==========
HTTP/1.1 404 Not Found
Content-Length: 14
Content-Type: text/plain
Connection: close

Page Not Found
```

Client

```
========== 2. GET /abc -> 404 Not Found ==========
[Client -> Server]
GET /abc HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 404 Not Found
Content-Length: 14
Content-Type: text/plain
Connection: close

Page Not Found
```

CASE3

Server

```
Client Connected : ('192.168.64.1', 50265)

========== REQUEST HEADER ==========
POST /message HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Length: 15
Content-Type: text/plain

========== REQUEST BODY ==========
hello professor

========== RESPONSE ==========
HTTP/1.1 201 Created
Content-Length: 32
Content-Type: text/plain
Connection: close

Message Created: HELLO PROFESSOR
```

Client

```
========== 3. POST /message -> 201 Created ==========
[Client -> Server]
POST /message HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Length: 15
Content-Type: text/plain

hello professor

[Server -> Client]
HTTP/1.1 201 Created
Content-Length: 32
Content-Type: text/plain
Connection: close

Message Created: HELLO PROFESSOR
```

CASE4

Server

```
Client Connected : ('192.168.64.1', 50268)

========== REQUEST HEADER ==========
POST /message HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Length: 0
Content-Type: text/plain

========== RESPONSE ==========
HTTP/1.1 400 Bad Request
Content-Length: 10
Content-Type: text/plain
Connection: close

Empty Body
```

Client

```
========== 4. POST /message empty body -> 400 Bad Request ==========
[Client -> Server]
POST /message HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Length: 0
Content-Type: text/plain

[Server -> Client]
HTTP/1.1 400 Bad Request
Content-Length: 10
Content-Type: text/plain
Connection: close

Empty Body
```

CASE5

Server

```
Client Connected : ('192.168.64.1', 50271)

========== REQUEST HEADER ==========
PUT /sample.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Type: application/octet-stream
Content-Length: 10

========== REQUEST BODY ==========
small file

========== RESPONSE ==========
HTTP/1.1 200 OK
Content-Length: 14
Content-Type: text/plain
Connection: close

Upload Success
```

Client

```
========== 5. PUT /sample.txt -> 200 OK ==========
[Client -> Server]
PUT /sample.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Type: application/octet-stream
Content-Length: 10

small file

[Server -> Client]
HTTP/1.1 200 OK
Content-Length: 14
Content-Type: text/plain
Connection: close

Upload Success
```

CASE6

Server

```
Client Connected : ('192.168.64.1', 50274)

========== REQUEST HEADER ==========
PUT /large.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Type: application/octet-stream
Content-Length: 100

========== REQUEST BODY ==========
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

========== RESPONSE ==========
HTTP/1.1 413 Payload Too Large
Content-Length: 14
Content-Type: text/plain
Connection: close

File Too Large
```

Client

```
========== 6. PUT /large.txt -> 413 Payload Too Large ==========
[Client -> Server]
PUT /large.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Type: application/octet-stream
Content-Length: 100

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

[Server -> Client]
HTTP/1.1 413 Payload Too Large
Content-Length: 14
Content-Type: text/plain
Connection: close

File Too Large
```

CASE7

Server

```
Client Connected : ('192.168.64.1', 50277)

========== REQUEST HEADER ==========
DELETE /sample.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

========== RESPONSE ==========
HTTP/1.1 200 OK
Content-Length: 14
Content-Type: text/plain
Connection: close

Delete Success
```

Client

```
========== 7. DELETE /sample.txt -> 200 OK ==========
[Client -> Server]
DELETE /sample.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 200 OK
Content-Length: 14
Content-Type: text/plain
Connection: close

Delete Success
```

CASE8

Server

```
Client Connected : ('192.168.64.1', 50280)

========== REQUEST HEADER ==========
DELETE /ghost.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

========== RESPONSE ==========
HTTP/1.1 404 Not Found
Content-Length: 14
Content-Type: text/plain
Connection: close

File Not Found
```

Client

```
========== 8. DELETE /ghost.txt -> 404 Not Found ==========
[Client -> Server]
DELETE /ghost.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 404 Not Found
Content-Length: 14
Content-Type: text/plain
Connection: close

File Not Found
```

CASE9

Server

```
Client Connected : ('192.168.64.1', 50283)

========== REQUEST HEADER ==========
DELETE /protected.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

========== RESPONSE ==========
HTTP/1.1 403 Forbidden
Content-Length: 14
Content-Type: text/plain
Connection: close

Protected File
```

Client

```
========== 9. DELETE /protected.txt -> 403 Forbidden ==========
[Client -> Server]
DELETE /protected.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 403 Forbidden
Content-Length: 14
Content-Type: text/plain
Connection: close

Protected File
```

CASE10

Server

```
Client Connected : ('192.168.64.1', 50286)

========== REQUEST HEADER ==========
HEAD / HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

========== RESPONSE ==========
HTTP/1.1 200 OK
Content-Length: 58
Content-Type: text/html
Connection: close
```

Client

```
========== 10. HEAD / -> 200 OK ==========
[Client -> Server]
HEAD / HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 200 OK
Content-Length: 58
Content-Type: text/html
Connection: close
```

□ 전체 소스 코드

client/client.py

```
import argparse
import socket
import time


# 서버 IP 와 포트 번호
# UTM 우분투 서버에서 ip addr 로 확인한 주소를 기본값으로 사용
SERVER_IP = "192.168.64.2"
SERVER_PORT = 8080
BUFFER_SIZE = 4096


def receive_response(client):
    # 서버가 Connection: close 로 연결을 닫을 때까지 응답을 계속 받음
    response_chunks = []

    while True:
        chunk = client.recv(BUFFER_SIZE)
        if not chunk:
            break
        response_chunks.append(chunk)

    return b"".join(response_chunks)


def send_request(request_bytes, host, port, title):
    # 하나의 테스트 케이스를 실행하는 함수
    # 요청 메시지를 출력하고, 서버로 보낸 뒤 응답 메시지도 출력
    print(f"\n========== {title} ==========")
    print("[Client -> Server]")
    print(request_bytes.decode(errors="replace").rstrip())

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
        client.connect((host, port))
        client.sendall(request_bytes)  # 요청 전체가 전송되도록 sendall 사용
        response = receive_response(client)

    print("\n[Server -> Client]")
    print(response.decode(errors="replace").rstrip())
    time.sleep(1)


def make_text_request(method, path, host, port, body=""):
    # GET, HEAD, POST, DELETE 처럼 문자열 body 를 사용하는 요청 생성
    # HTTP Request Line + Header + 빈 줄 + Body 순서로 구성
    request = (
        f"{method} {path} HTTP/1.1\r\n"
        f"Host: {host}:{port}\r\n"
        "User-Agent: CN-Assignment-Client/1.0\r\n"
        "Connection: close\r\n"
    )

    if method in ("POST", "PUT"):
        body_bytes = body.encode()
        # body 가 있는 메서드는 Content-Length 를 넣어 서버가 body 크기를 알 수 있게 함
        request += f"Content-Length: {len(body_bytes)}\r\n"
        request += "Content-Type: text/plain\r\n"
    else:
        body_bytes = b""

    request += "\r\n"
    return request.encode() + body_bytes
```

```python
def make_binary_request(method, path, host, port, body):
    # PUT 요청처럼 bytes 데이터를 보내는 경우 사용
    header = (
        f"{method} {path} HTTP/1.1\r\n"
        f"Host: {host}:{port}\r\n"
        "User-Agent: CN-Assignment-Client/1.0\r\n"
        "Connection: close\r\n"
        "Content-Type: application/octet-stream\r\n"
        f"Content-Length: {len(body)}\r\n"
        "\r\n"
    ).encode()

    return header + body



def get_cases(host, port):
    # 과제에서 확인할 HTTP Method / Response 테스트 케이스들
    return {
        "1": (
            "GET 200",
            lambda: send_request(
                make_text_request("GET", "/", host, port),
                host,
                port,
                "1. GET / -> 200 OK",
            ),
        ),
        "2": (
            "GET 404",
            lambda: send_request(
                make_text_request("GET", "/abc", host, port),
                host,
                port,
                "2. GET /abc -> 404 Not Found",
            ),
        ),
        "3": (
            "POST 201",
            lambda: send_request(
                make_text_request("POST", "/message", host, port, "hello professor"),
                host,
                port,
                "3. POST /message -> 201 Created",
            ),
        ),
        "4": (
            "POST 400",
            lambda: send_request(
                make_text_request("POST", "/message", host, port, ""),
                host,
                port,
                "4. POST /message empty body -> 400 Bad Request",
            ),
        ),
        "5": (
            "PUT 200",
            lambda: send_request(
                make_binary_request("PUT", "/sample.txt", host, port, b"small file"),
                host,
                port,
                "5. PUT /sample.txt -> 200 OK",
            ),
        ),
        "6": (
            "PUT 413",
            lambda: send_request(
                make_binary_request("PUT", "/large.txt", host, port, b"A" * 100),
                host,
                port,
                "6. PUT /large.txt -> 413 Payload Too Large",
            ),
        ),
        "7": (
            "DELETE 200",
            lambda: send_request(
                make_text_request("DELETE", "/sample.txt", host, port),
                host,
```

```
                port,
                "7. DELETE /sample.txt -> 200 OK",
            ),
        ),
        "8": (
            "DELETE 404",
            lambda: send_request(
                make_text_request("DELETE", "/ghost.txt", host, port),
                host,
                port,
                "8. DELETE /ghost.txt -> 404 Not Found",
            ),
        ),
        "9": (
            "DELETE 403",
            lambda: send_request(
                make_text_request("DELETE", "/protected.txt", host, port),
                host,
                port,
                "9. DELETE /protected.txt -> 403 Forbidden",
            ),
        ),
        "10": (
            "HEAD 200",
            lambda: send_request(
                make_text_request("HEAD", "/", host, port),
                host,
                port,
                "10. HEAD / -> 200 OK",
            ),
        ),
    }



def print_menu(cases):
    # 사용자가 원하는 테스트 케이스를 선택할 수 있도록 메뉴 출력
    print("\n========== MENU ==========")
    for key, (title, _) in cases.items():
        print(f"{key}. {title}")
    print("a. RUN ALL")
    print("0. EXIT")



def main():
    # 실행할 때 --host, --port 옵션으로 서버 주소를 바꿀 수 있게 함
    parser = argparse.ArgumentParser(description="TCP socket HTTP client")
    parser.add_argument("--host", default=SERVER_IP, help="server IP address")
    parser.add_argument("--port", type=int, default=SERVER_PORT, help="server port")
    args = parser.parse_args()

    cases = get_cases(args.host, args.port)

    while True:
        print_menu(cases)
        menu = input("Select : ").strip().lower()

        if menu == "0":
            break
        if menu == "a":
            # a 를 입력하면 전체 케이스를 순서대로 실행
            for _, run_case in cases.values():
                run_case()
            continue
        if menu in cases:
            cases[menu][1]()
        else:
            print("Wrong Menu")



if __name__ == "__main__":
    main()
```

server/server.py

```
import argparse
import os
```

```python
import socket


# 서버는 모든 네트워크 인터페이스에서 8080 포트로 대기
HOST = "0.0.0.0"
PORT = 8080
BUFFER_SIZE = 4096
MAX_UPLOAD_SIZE = 30  # 이 크기를 넘으면 413 응답을 보내기 위한 기준


# 서버 파일 위치를 기준으로 www, uploads 폴더 경로 설정
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
UPLOAD_DIR = os.path.join(BASE_DIR, "uploads")
WWW_DIR = os.path.join(BASE_DIR, "www")
INDEX_FILE = os.path.join(WWW_DIR, "index.html")



def prepare_files():
    # 서버 실행에 필요한 폴더와 기본 파일 준비
    os.makedirs(UPLOAD_DIR, exist_ok=True)
    os.makedirs(WWW_DIR, exist_ok=True)

    protected_file = os.path.join(UPLOAD_DIR, "protected.txt")
    if not os.path.exists(protected_file):
        with open(protected_file, "w", encoding="utf-8") as file:
            file.write("This file is protected.")



def recv_until_header_end(conn):
    # HTTP Header 는 \r\n\r\n 에서 끝나므로 그 지점까지 먼저 수신
    data = b""

    while b"\r\n\r\n" not in data:
        chunk = conn.recv(BUFFER_SIZE)
        if not chunk:
            break
        data += chunk

    return data



def parse_headers(header_text):
    # Request Line 에서 method, path, version 을 분리
    lines = header_text.split("\r\n")
    method, path, version = lines[0].split()

    # 나머지 Header 들은 딕셔너리로 저장
    headers = {}
    for line in lines[1:]:
        if ":" in line:
            name, value = line.split(":", 1)
            headers[name.strip().lower()] = value.strip()

    return method, path, version, headers



def recv_body(conn, body_start, content_length):
    # Content-Length 에 적힌 크기만큼 Body 를 끝까지 수신
    body = body_start

    while len(body) < content_length:
        chunk = conn.recv(BUFFER_SIZE)
        if not chunk:
            break
        body += chunk

    return body[:content_length]
```

```python
def make_response(status_line, body=b"", content_type="text/plain", include_body=True):
    # HTTP Response 는 Status Line + Header + 빈 줄 + Body 순서로 구성
    headers = [
        f"Content-Length: {len(body)}",
        f"Content-Type: {content_type}",
        "Connection: close",
    ]
    response = status_line + "\r\n" + "\r\n".join(headers) + "\r\n\r\n"
    response_bytes = response.encode("utf-8")

    if include_body:
        response_bytes += body

    return response_bytes


def get_static_file_path(path):
    # URL 경로를 server/www 폴더 안의 실제 파일 경로로 바꿔줌
    # / 요청은 기본 페이지인 www/index.html 로 처리
    if path == "/":
        return INDEX_FILE

    request_path = path.lstrip("/")
    file_path = os.path.join(WWW_DIR, request_path)

    # 폴더 경로로 요청한 경우에는 그 안의 index.html 을 기본 파일로 사용
    if os.path.isdir(file_path):
        file_path = os.path.join(file_path, "index.html")

    return file_path


def handle_get(path):
    # GET 요청 경로에 해당하는 파일이 www 폴더 안에 있으면 200 OK 반환
    file_path = get_static_file_path(path)
    if file_path and os.path.exists(file_path) and os.path.isfile(file_path):
        with open(file_path, "rb") as file:
            body = file.read()
        return make_response("HTTP/1.1 200 OK", body, "text/html")

    # 파일이 없으면 없는 페이지로 처리
    body = b"Page Not Found"
    return make_response("HTTP/1.1 404 Not Found", body)


def handle_head(path):
    # HEAD 는 GET 과 비슷하지만 Body 는 보내지 않음
    file_path = get_static_file_path(path)
    if file_path and os.path.exists(file_path) and os.path.isfile(file_path):
        with open(file_path, "rb") as file:
            body = file.read()
        return make_response("HTTP/1.1 200 OK", body, "text/html", include_body=False)

    body = b"Page Not Found"
    return make_response("HTTP/1.1 404 Not Found", body, include_body=False)


def handle_post(path, body):
    # POST 는 /message 경로만 처리
    if path != "/message":
        return make_response("HTTP/1.1 404 Not Found", b"Page Not Found")

    # Body 가 비어 있으면 잘못된 요청으로 판단
    if not body.strip():
        return make_response("HTTP/1.1 400 Bad Request", b"Empty Body")
```

```python
    # 정상 Body 가 오면 생성 성공으로 201 응답
    response_body = b"Message Created: " + body.upper()
    return make_response("HTTP/1.1 201 Created", response_body)



def handle_put(path, body):
    # PUT 으로 받은 데이터는 uploads 폴더에 파일로 저장
    filename = os.path.basename(path)

    if path not in ("/sample.txt", "/large.txt"):
        return make_response("HTTP/1.1 404 Not Found", b"Upload Path Not Found")

    # 크기가 너무 크면 413 응답
    if len(body) > MAX_UPLOAD_SIZE:
        return make_response("HTTP/1.1 413 Payload Too Large", b"File Too Large")

    upload_path = os.path.join(UPLOAD_DIR, filename)
    with open(upload_path, "wb") as file:
        file.write(body)

    return make_response("HTTP/1.1 200 OK", b"Upload Success")



def handle_delete(path):
    # DELETE 요청은 uploads 폴더 안의 파일을 대상으로 처리
    filename = os.path.basename(path)

    # protected.txt 는 삭제 금지 파일로 설정
    if filename == "protected.txt":
        return make_response("HTTP/1.1 403 Forbidden", b"Protected File")

    filepath = os.path.join(UPLOAD_DIR, filename)

    if not os.path.exists(filepath):
        return make_response("HTTP/1.1 404 Not Found", b"File Not Found")

    os.remove(filepath)
    return make_response("HTTP/1.1 200 OK", b"Delete Success")



def handle_client(conn, addr):
    # 클라이언트 하나의 요청을 처리하는 부분
    print(f"\nClient Connected : {addr}")

    try:
        request = recv_until_header_end(conn)
        if not request:
            return

        header_bytes, body_start = request.split(b"\r\n\r\n", 1)
        header_text = header_bytes.decode(errors="replace")
        method, path, version, headers = parse_headers(header_text)
        content_length = int(headers.get("content-length", "0"))
        body = recv_body(conn, body_start, content_length)

        print("\n========== REQUEST HEADER ==========")
        print(header_text)
        if body:
            print("\n========== REQUEST BODY ==========")
            print(body.decode(errors="replace"))


        # Method 에 따라 각각의 처리 함수로 분기
        if method == "GET":
            response = handle_get(path)
        elif method == "HEAD":
            response = handle_head(path)
        elif method == "POST":
            response = handle_post(path, body)
        elif method == "PUT":
```

```python
            response = handle_put(path, body)
        elif method == "DELETE":
            response = handle_delete(path)
        else:
            response = make_response("HTTP/1.1 405 Method Not Allowed", b"Method Not Allowed")

        conn.sendall(response)

        print("\n========== RESPONSE ==========")
        print(response.decode(errors="replace").rstrip())

    except Exception as error:
        response = make_response("HTTP/1.1 400 Bad Request", f"Bad Request: {error}".encode())
        conn.sendall(response)
        print(f"Error: {error}")
    finally:
        conn.close()


def main():
    # --host, --port 옵션을 사용하면 실행할 주소와 포트를 바꿀 수 있음
    parser = argparse.ArgumentParser(description="TCP socket HTTP server")
    parser.add_argument("--host", default=HOST, help="server bind address")
    parser.add_argument("--port", type=int, default=PORT, help="server port")
    args = parser.parse_args()

    prepare_files()

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        # 서버를 바로 재실행할 때 포트가 잠겨있는 문제를 줄이기 위한 옵션
        server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server.bind((args.host, args.port))
        server.listen(5)

        print(f"HTTP Server Started ({args.host}:{args.port})")

        try:
            while True:
                conn, addr = server.accept()
                handle_client(conn, addr)
        except KeyboardInterrupt:
            print("\nHTTP Server Stopped")


if __name__ == "__main__":
    main()
```